deep-MARLIN: Using Deep Multi-Agent Reinforcement Learning for Adaptive Traffic Light Control

Robin Klöckner
Goethe University
Frankfurt, Germany

Patrick Klose
Goethe University
Frankfurt, Germany

ABSTRACT

Almost every major city in the world is facing a significant economic loss caused by traffic congestions. In this context, it already has been shown that *Adaptive Traffic Light Control (ATLC)* can be an effective solution to improve a diversity of different traffic-related metrics. The problem of ATLC can be modeled in various ways with reinforcement learning being one of the most promising frameworks. Especially the application of *Multi-Agent Reinforcement Learning (MARL)* can be a suitable approach for learning to adaptively control the traffic of realistic road networks. Among the set of MARL algorithms, *Multi-Agent Reinforcement Learning for Integrated Network (MARLIN)* stands out and is shown to be particularly suited to the problem of ATLC with producing remarkable results. MARLIN models the multi-agent framework as a stochastic game providing an explicit coordination mechanism for the agents. However, in MARLIN, the possible size of the state and action space is limited and the features for the state representation need to be hand-crafted. Therefore, in this study, the algorithm is combined with function approximation by using artificial neural networks to overcome these limitations. *deep-MARLIN* is explained and benchmarked in a large and realistic traffic environment using *Simulation of Urban MObility (SUMO)*. The results indicate that when compared to MARLIN, deep-MARLIN converges faster to a policy that is producing lower average vehicle delay.

CCS CONCEPTS

• **Computing methodologies** → **Multi-agent reinforcement learning**.

KEYWORDS

Artificial Intelligence, Machine Learning, Reinforcement Learning, Neural Networks, Multi-Agent, Game Theory, Stochastic Game, Adaptive Traffic Light Control

ACM Reference Format:
Robin Klöckner and Patrick Klose. 2020. deep-MARLIN: Using Deep Multi-Agent Reinforcement Learning for Adaptive Traffic Light Control. In *Proceedings of the 3rd International Conference on Applications of Intelligent Systems (APPIS 2020), January 7–9, 2020, Las Palmas de Gran Canaria, Spain.* ACM, New York, NY, USA, 6 pages. https://doi.org/10.1145/3378184.3378194

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than ACM must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
APPIS 2020, January 7–9, 2020, Las Palmas de Gran Canaria, Spain
© 2020 Association for Computing Machinery.
ACM ISBN 978-1-4503-7630-3/20/01…$15.00
https://doi.org/10.1145/3378184.3378194

1 INTRODUCTION

Almost every major city in the world is facing a significant economic loss caused by traffic congestions. The reasons for these congestions are manifold and include, among other things, the increasing population in urban areas and that it is nearly impossible to build new roads in such areas due to limited space [9]. An alternative solution can be the improvement of a city's traffic light system. In *Adaptive Traffic Light Control (ATLC)*, traffic lights are controlled in real-time based on the traffic situation, which already has been shown to reduce traffic congestions effectively [6].

Modeling the problem of ATLC based on *Reinforcement Learning (RL)* seems to be an especially suitable approach, since a traffic light that is changing signals depending on the traffic situation corresponds to the idea of RL, in which an agent *learns* to control its environment by interacting with it [23]. However, using RL as the underlying framework of ATLC can be challenging when state and action spaces are large, i.e., when the (simulated) traffic environment is getting more realistic. Furthermore, to properly capture the dynamics of large traffic environments, it can be a good idea to model the problem of ATLC based on a framework providing multiple co-existing agents. However, this can significantly increase the complexity of the problem, since the behavior of one agent may influence the behavior of other agents in the environment.

To tackle all these problems, various methods have been developed, ranging from function approximation in *Single-Agent Reinforcement Learning (SARL)* [8], to (coordinated) *Multi-Agent Reinforcement Learning (MARL)* [6]. This evolution also led to the development of *Multi-Agent Reinforcement Learning for Integrated Network (MARLIN)* [6] with producing remarkable results in the field of ATLC. MARLIN provides a multi-agent setting modeled as a stochastic game which incorporates an explicit coordination mechanism between the agents to (partly) capture the non-stationarity in the environment which may be imposed by the existence of multiple agents [4, 14]. However, MARLIN relies on a table-based representation limiting its applicability when the road network is getting more realistic. In contrast, alternative ATLC algorithms which already combine MARL and function approximation, neglect explicit coordination between the agents.

Therefore, in this study, a MARL algorithm is developed incorporating function approximation as well as an explicit coordination mechanism, in order to exploit all the advantages that come along with the use of function approximation while also taking into account the instability which may be imposed by multiple agents. Since MARLIN provides a theoretically well-developed fundament, it is used as the basis and its table-based representation is replaced by using *Artificial Neural Networks (ANNs)*. The proposed algorithm for which we refer to as *deep-MARLIN* is then applied to an ATLC problem with a traffic environment based on a large and realistic

road network having 41 traffic light controlled junctions and which is simulated based on *Simulation of Urban MObility (SUMO)* [15]. The study concludes by showing that when compared to MARLIN, deep-MARLIN can not only converge faster but it can also provide a policy controlling the vehicles better when measured in average vehicle delay.

2 LITERATURE REVIEW

Related work to the problem of ATLC ranges from SARL to frameworks using multiple RL agents. The authors of [1] used tabular Q-learning [25] to control traffic lights of a single junction. However, in tabular Q-learning, the state of the environment can usually only be represented by a few hand-crafted features (e.g. queue length and cumulative delay [1, 6, 7]). This bears the risk that they do not represent the environment in an adequate way. To overcome this limitation, the authors of [8] approximated the state-action value function by an ANN. However, the traffic environment of both studies only provided a single junction, which does not reflect the characteristics of a realistic road network.

In the study of [26], the author used MARL to control traffic lights in an environment with multiple junctions. Vehicle-based value functions and a dynamic programming-like algorithm with three different communication settings were used to minimize the cumulative waiting time of all vehicles at a traffic light. Even if the proposed algorithm outperformed a pre-timed control algorithm, a simulated environment consisting out of six traffic light controlled junctions is still too simple in order to represent the characteristics of a realistic road network. The authors of [11] extended the work of [26] by incorporating explicit coordination under the assumption that an agent is only affected by its neighboring agents rather than by all agents in the road network. This was done by using coordination graphs and the max-plus algorithm. The results indicate that the proposed algorithm can improve efficiency. However, the model-based algorithms of [26] and [11] are computationally demanding when compared to model-free RL methods such as tabular Q-learning.

Therefore, the authors of [2] developed a model-free MARL algorithm with non-linear function approximation to control traffic lights in a road network with five intersections. However, only the central traffic light was able to learn while the outbound traffic lights only controlled the traffic by giving right-of-way to the lanes with the longest queues. Furthermore, the layout of all the intersections were equal, which is a huge simplification when compared to a realistic road network.

In [6], the authors developed MARLIN which represents a MARL algorithm based on the concept of a stochastic game enabling explicit coordination between the agents. The algorithm incorporates the concept of modular Q-learning [21] and the principles of locality of interaction [20] and was applied to a large-scale traffic control problem, in which agents learned optimal behavior in a decentralized manner. The results indicate that MARLIN outperformed the baseline algorithm (signal timing sheet) by reducing the average intersection delay by up to 38 percent.

In a recent study, the authors of [5] proposed *MA2C* which represents a modified version of *Asynchronous Advantage Actor Critic (A3C)* [17] by synchronizing the algorithm (A2C) and by extending it to multiple independent agents (IA2C) which are acting in a partial observable environment with limited communication abilities. MA2C was applied to a large-scale ATLC problem in which each intersection was controlled by a separate RL agent. The results indicate a superior performance of MA2C over IA2C and over a Greedy policy. However, even when the communicated policies of an agent's neighbors are taken into account, the agents do not explicitly coordinate the action-selection.

3 THEORETICAL BACKGROUND

In classical SARL, a problem is often formalized as a finite *Markov Decision Process (MDP)* in which an agent learns by continually interacting with an environment $\mathcal{E}$ in discrete time steps t to achieve a goal [23]. Every time step t, the agent receives a representation of $\mathcal{E}$, called a state $S_t \in \mathcal{S}$, with $\mathcal{S}$ being the finite set of all possible states. Based on the current state, the agent then selects an action $A_t \in \mathcal{A}(S_t)$, whereas $\mathcal{A} : \mathcal{S} \rightarrow P(\mathcal{A}^+) \setminus \emptyset$ is a function that maps a state to an element of the power set P of $\mathcal{A}^+$, with $\mathcal{A}^+$ being the finite set of all possible actions in $\mathcal{E}$. The stochastic function $\pi(. \mid S_t) : \mathcal{A}(S_t) \rightarrow \mathbb{R}^{\geq 0}$ controls which specific action to choose at time step t, whereby $\sum_{a \in \mathcal{A}(S_t)} \pi(a \mid S_t) = 1$ always holds.

After choosing a specific action A_t at time step t, the state of the environment advances from S_t to $S_{t+1} \in \mathcal{S}$ and the agent receives a reward $R_{t+1} \in \mathcal{R} \subset \mathbb{R}$, with $\mathcal{R}$ being the finite set of all possible rewards. All these dynamics are described by the stochastic environment function $\mathcal{P}(., . \mid S_t, A_t) : \mathcal{S} \times \mathcal{R} \rightarrow \mathbb{R}^{\geq 0}$, whereby $\sum_{s \in \mathcal{S}} \sum_{r \in \mathcal{R}} \mathcal{P}(s, r \mid S_t, A_t) = 1$ always holds. If all of these so called conditional transition probabilities only depend on the recent state-action pair, the states of the environment fulfill the Markov property, which is a necessary assumption of many RL algorithms [23].

Often, the agent's formal goal is to maximize the return G_0, whereby G_t is defined as a function over the sequence of rewards from time step t to T, i.e., $G_t := f(R_t, R_{t+1}, ..., R_T)$ [23]. This can be achieved by finding an optimal policy, denoted as $\pi^* := \arg\max_{\pi} \mathbb{E}_{S_{i \geq 0}, R_{j \geq 1} \sim \mathcal{P}, A_{k \geq 0} \sim \pi}(G_1)$, where the starting state S_0 is drawn from the start distribution $\mathcal{P}$ over $\mathcal{S}$ [12].

Let $Q_\pi(S_t, A_t) := \mathbb{E}_{S_{i>t}, R_{j>t} \sim \mathcal{P}, A_{k>t} \sim \pi}(G_{t+1} \mid S_t, A_t)$ be a state-action value function representing the expected return of taking action A_t in state S_t and following policy π thereafter. An optimal policy π^* is then often determined by using a form of *Generalized Policy Iteration (GPI)*. Based on GPI, two interconnected processes are taking place, i.e., $Q_\pi(S_t, A_t)$ is updated to be consistent with the current policy π (policy evaluation) and π is optimized on the basis of $Q_\pi(S_t, A_t)$ (policy improvement) [23].

One well-known method of GPI is tabular Q-learning in which the sample episodes are generated according to a behavior policy π^b, while the target policy π is learned. However, when the state space $\mathcal{S}$ is too large, tabular Q-learning becomes unsuitable. To handle large state spaces, the state-action value function can also be approximated, e.g., by ANNs [23]. In the study of [18], the authors present *Deep Q-learning (DQL)* in which the state-action value of a given state-action pair is represented by an ANNs output unit.

When ANNs are used for function approximation, RL algorithms can suffer from instability and tend to diverge [19, 24]. One reason for this is that the sequence of observations used to train the ANN

is correlated in a timely manner [18, 19]. Furthermore, the target function in the update rule is non-stationary because the behavior policy may change significantly with every update of the ANN and therefore may change the data distribution [18, 19]. To counteract these problems, DQL applies *experience replay* [13].

Experience replay removes the timely correlation between the observations to some extend and changes in the data distribution are smoothed out, which in turn reduces the variance in the updates for the weights of the ANN [18, 19]. To apply experience replay, at every time step t, the agent's experience represented by the tuple $(S_t, A_t, R_{t+1}, S_{t+1})$ is stored to a replay memory $\mathcal{M}$. Minibatches of these transitions are then usually uniformly sampled and used to train the ANN [18].

In the MDP formalization of SARL, an agent's environment is often assumed to be stationary. Therefore, other agents can only be part of the environment when having a fixed behavior [14]. However, multiple adaptive agents interacting with the same environment can cause (further) non-stationarity in the environment [4]. This non-stationarity can to some extend be captured by MARL frameworks with a coordination mechanism [4]. In this context, a promising approach is to frame the MARL problem as a stochastic game [22], which is a concept from game theory where the Markov property holds.

In this context, MARLIN is a MARL algorithm in which a general-sum stochastic game is used as the underlying framework. The principles of locality of interaction and the concept of modular Q-learning are incorporated to reduce the number of joint state-action value functions and the size of the state space. MARLIN provides an explicit coordination mechanism by estimating the policies of the agents' neighbors (called *models*), which are then used to determine an optimal action. This best response action, which may not be unique, is derived from the values of the joint state-action values, each weighted by the corresponding model value [6]. When each agent i (stochastically) chooses its best response action, given the (stochastically chosen) best response actions of the other agents, the behavior of the agents form a (mixed) *Nash Equilibrium (NE)* which is the solution concept of stochastic games [3, 6].

Algorithm 1: deep-MARLIN

1 Finite set of agents: $\mathcal{N}$
2 Finite set of neighbors of agent $i \in \mathcal{N}$: $\mathcal{J}_i$
3 Initialize weights $\theta^{\omega}_{i,j,0} \sim N(\mu, \sigma^2), \forall i \in \mathcal{N}, j \in \mathcal{J}_i$
4 Initialize weights $\theta^{Q}_{i,j,0} \sim N(\mu, \sigma^2), \forall i \in \mathcal{N}, j \in \mathcal{J}_i$
5 Initialize replay memories $\mathcal{M}^{\omega_{i,j}}, \forall i \in \mathcal{N}, j \in \mathcal{J}_i$
6 Initialize replay memories $\mathcal{M}^{Q_{i,j}}, \forall i \in \mathcal{N}, j \in \mathcal{J}_i$
7 Initialize learning rates $\alpha, \beta \in (0, 1]$
8 Initialize discount factor $\gamma \in [0, 1]$
9 (Randomly) initialize $S_{i,0}, A_{i,0}, \forall i \in \mathcal{N}$
10 **for** *each time step t* **do**
11 Execute the global joint action A_t
12 **for** *each agent* $i \in \mathcal{N}$ **do**
13 **for** *each neighbor* $j \in \mathcal{J}_i$ **do**
14 Observe $A_{j,t}, S_{i,t+1}, S_{j,t+1}, R_{i,t+1}$
15 Store $(S_{i,t}, S_{j,t}, A_{j,t})$ to $\mathcal{M}^{\omega_{i,j}}$
16 Sample minibatch X^{ω} from $\mathcal{M}^{\omega_{i,j}}$
17 Update model-network parameters by using X^{ω}: $\theta^{\omega}_{i,j,t+1} \leftarrow \theta^{\omega}_{i,j,t} + \alpha \nabla_{\theta^{\omega}_{i,j}} L^{\omega}_t(\theta^{\omega}_{i,j,t})$
18 Store $(S_{i,t}, S_{j,t}, A_{i,t}, A_{j,t}, R_{i,t+1}, S_{i,t+1}, S_{j,t+1})$ to $\mathcal{M}^{Q_{i,j}}$
19 Sample minibatch X^{Q} from $\mathcal{M}^{Q_{i,j}}$
20 Update Q-network parameters by using X^{Q}: $\theta^{Q}_{i,j,t+1} \leftarrow \theta^{Q}_{i,j,t} + \beta \nabla_{\theta^{Q}_{i,j}} L^{Q}_t(\theta^{Q}_{i,j,t})$
21 Choose action $A_{i,t+1} \in \mathcal{A}_i(S_{i,t+1})$ using π^b_i, whereas the best response action is determined by $A^{br}_{i,t+1} = \arg \max_{a_i \in \mathcal{A}_i(S_{i,t+1})} \sum_{j \in \mathcal{J}_i} \sum_{a_j \in \mathcal{A}_j(S_{j,t+1})} [\hat{Q}((S_{i,t+1}, S_{j,t+1}, a_i, a_j) \mid \theta^{Q}_{i,j,t+1}) \hat{\omega}_{a_j}((S_{i,t+1}, S_{j,t+1}) \mid \theta^{\omega}_{i,j,t+1})]$
22 $S_t = S_{t+1}$

4 DEEP-MARLIN

The proposed algorithm is referred to as *deep-MARLIN* (Algorithm 1). It is based on MARLIN but the table-based representations of the joint state-action value functions and model functions are replaced by ANNs. For every neighboring agent pair i and j, $\theta^{Q}_{i,j,t}$ denotes the parameters (weights) of the corresponding Q-network and $\theta^{\omega}_{i,j,t}$ denotes the parameters of the corresponding model-network, both at time step t.

Every time step t, each agent observes its local environment and plays a game with each of its neighboring agents in order to learn (update) its own approximated joint state-action value function and model function. These are then used to coordinate the action selection in form of a best response to the expected behavior of its neighboring agents. While doing so, observed new states, rewards and actions are saved as a tuple to the replay memory $\mathcal{M}^{\omega_{i,j}}$ for the model-network (line 15) and to $\mathcal{M}^{Q_{i,j}}$ for the Q-network (line 18). Minibatches X^{ω} and X^{Q} are then uniformly sampled from the replay memories (line 16 and 19) and are used to train the model-network and the Q-network, respectively. In doing so, the loss function (mean squared error) of the model-network $L^{\omega}_t(\theta^{\omega}_{i,j,t})$ is calculated based on the already experienced transitions $x \in X^{\omega}$:

$$L^{\omega}_t(\theta^{\omega}_{i,j,t}) := \frac{1}{|X^{\omega}|} \sum_{x \in X^{\omega}} |y_k - \hat{\omega}(x' \mid \theta^{\omega}_{i,j,t})|^2,$$

where $x = (S_{i,k}, S_{j,k}, A_{j,k})$, $x' = (S_{i,k}, S_{j,k})$ and with y_k being a one-hot vector having the value of 1 at the position corresponding to $A_{j,k}$ as well as k being the time step of the corresponding transition. The loss function (mean squared error) of the Q-network $L^{Q}_t(\theta^{Q}_{i,j,t})$ is calculated based on the experienced transitions $x \in X^{Q}$:

$$L^{Q}_t(\theta^{Q}_{i,j,t}) := \frac{1}{|X^{Q}|} \sum_{x \in X^{Q}} [R_{i,k+1} + \gamma \mathcal{B}_{i,k+1}(\mathcal{A}_j) - \hat{Q}(x' \mid \theta^{Q}_{i,j,t})]^2,$$

where $x = (S_{i,k}, S_{j,k}, A_{i,k}, A_{j,k}, R_{i,k+1}, S_{i,k+1}, S_{j,k+1})$ and $x' = (S_{i,k}, S_{j,k}, A_{i,k}, A_{j,k})$. Furthermore, $\mathcal{B}_{i,k+1}(\mathcal{A}_j)$ is defined to be

$$\mathcal{B}_{i,k+1}(\mathcal{A}_j) \coloneqq \max_{a_i \in \mathcal{A}_i(S_{i,k+1})} \sum_{a_j \in \mathcal{A}_j(S_{j,k+1})} [\hat{Q}((S_{i,k+1}, S_{j,k+1}, a_i, a_j) \mid \theta^{Q}_{i,j,t}) \hat{\omega}_{a_j}((S_{i,k+1}, S_{j,k+1}) \mid \theta^{\omega}_{i,j,t+1})],$$

where $\hat{\omega}_{a_j}$ represents the value of $\hat{\omega}$ at the position corresponding to a_j. $\mathcal{B}_{i,k+1}(\mathcal{A}_j)$ represents the best response state-action value that replaces the maximum state-action value in the target of standard tabular Q-learning [23].

The model-network parameters $\theta^{\omega}_{i,j,t}$ as well as the Q-network parameters $\theta^{Q}_{i,j,t}$ are then updated in line 17 and 20, respectively. Coordinated decision making takes place indirectly in line 21 by determining the best response action $A^{br}_{i,t+1}$. An agent's best response action in turn is the action that maximizes the sum of all joint state-action values of the corresponding neighbors and their possible actions, whereas each joint state-action value is weighted by the corresponding model value.

5 SIMULATION ENVIRONMENT

In order to investigate the performance of deep-MARLIN, the algorithm was applied to a traffic environment simulated based on SUMO. A part of Frankfurt am Main's road network was downloaded from *OpenStreetMap*[1]. This city was chosen due to its historical grown and rather complex road network which was adjusted in a preprocessing step to meet the required conditions. The final road network (Figure 1) was a simplified copy of Frankfurt am Main's city center consisting out of 41 traffic light controlled junctions with different layouts and different number of phases. Each lane approaching a traffic light was equipped with a lanearea detector[2] (E2; 70 meters long at maximum), which allowed vehicle detection similar to traffic surveillance cameras. Due to SUMO's vehicle teleporting approach, the detectors may also have recognized vehicles which were actually not on the detector which in turn let the data being (partly) noisy. The number of vehicles in the road network was held constant at 400 and every vehicle traveled a minimum distance of 450 meters (see Table 1 for further parameters).

The algorithm was implemented in the main file written in Python programming language. The *TraCI* module from SUMO was used to retrieve values from the simulation environment and to manipulate the traffic lights. After receiving data from SUMO (frequency: 1s), the simulation stopped until the data was processed and until new commands from the agents were sent back to SUMO.

Agents were able to take two different modes. In the active mode, agents interacted with the environment as described in Algorithm 1. An agent in turn was in active mode, if it was allowed to perform an action at the current time step, that is, the minimum elapsed time of the current phase had been achieved (five seconds after the phase had been changed and one second after the current phase had been extended). The action space of each agent consisted of its possible phases, either two or three. The agent could either extend the current phase by one second or it could choose one of the other

[1] https://www.openstreetmap.org
[2] https://sumo.dlr.de/docs/Simulation/Output/Lanearea_Detectors_(E2).html



Figure 1: The realistic road network

phases. If a phase different from the current phase was chosen, the agent was switched into inactive mode. In this case, the main file automatically sent pre-defined phase changing signals to SUMO, i.e., yellow signals for three seconds, followed by red signals for two seconds before signalizing the new phase with a minimum phase duration of five seconds. In the inactive mode, the agents did nothing but updating the elapsed time of the current phase.

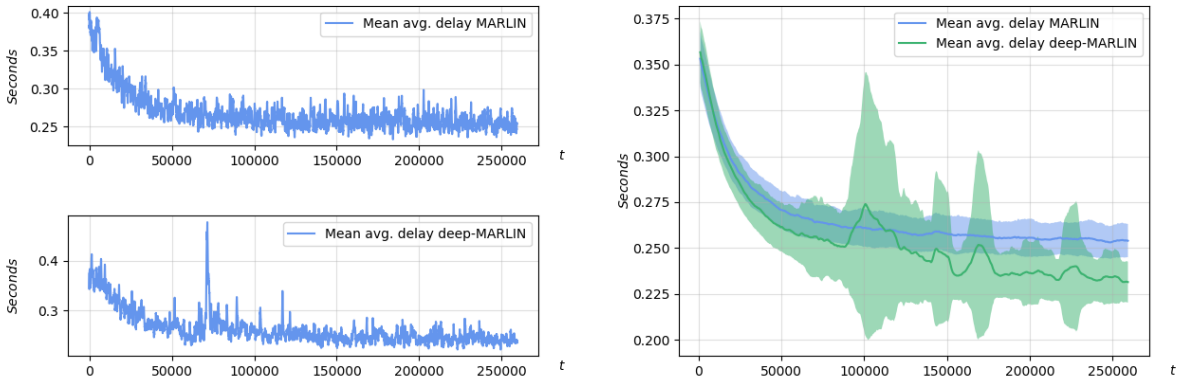
The state of an agent i was represented by a vector with $2+2*|\mathcal{E}_i|$ components, where $\mathcal{E}_i$ is the finite set of all detectors corresponding to agent i. The first component was the current phase in binary representation and the second component was the elapsed time of this phase. The remaining $2 * |\mathcal{E}_i|$ components represented traffic information from the detectors $E_l \in \mathcal{E}_i$ (two values per detector), whereby E_l denotes the detector of lane l from the set of all detectors of agent i. Let $\mathcal{V}$ be the set of all vehicles in the road network and let $v \in C_{E_l,t} \subseteq \mathcal{V}$ be a vehicle that is within the detection area of E_l at time step t. Based on this definition, the first detector value represented the number of vehicles $|C_{E_l,t}|$ which were located on the detector E_l at time step t, regardless of their current speed. The second component represented the queue length on the detector as the number of delayed vehicles, whereas a vehicle was declared as delayed if its speed was below two meters per second.

The delay D of vehicle v at time step $t+1$ ($D_{v,t+1}$) and the cumulative delay of all vehicles within the detection area of a lane l approaching agent i at time step $t+1$ ($D_{E_l,t+1}$) were defined as:

$$D_{v,t+1} \coloneqq \begin{cases} D_{v,t} + 1 & v \in C_{E_l,t+1} \text{ and its speed is } < 2\frac{m}{s} \\ D_{v,t} & v \in C_{E_l,t+1} \text{ and its speed is } \geq 2\frac{m}{s} \\ 0 & v \in C_{E_l,t+1} \wedge v \notin C_{E_l,t} \text{ (new vehicle)} \\ 0 & v \notin C_{E_l,t+1} \end{cases}$$

$$D_{E_l,t+1} \coloneqq \sum_{v \in C_{E_l}} D_{v,t+1}$$

The reward function of agent i was then defined as $R_{i,t+\eta} \coloneqq \sum_{E_l \in \mathcal{E}_i}(D_{E_l,t} - D_{E_l,t+\eta})$, i.e., the difference in total cumulative delay of all vehicles within all detection areas of agent i between two decision points, whereas η is the elapsed time from the last decision point at time step t.



(a) Average delay of MARLIN and deep-MARLIN

(b) Smoothed average delay of 20 instances

Figure 2: Comparing MARLIN and deep-MARLIN

Due to limited computational capabilities, all agents used the same two ANNs, one to approximate the joint state-action values (Q-network) and one to approximate the model values (model-network). Therefore, the input vectors for both networks were always augmented by the corresponding agent ID in binary notation. Architecturial details can be found in Table 2 and 3.

Each agent selected, based on its behavior policy π_i^b, the best response action $A_{i,t}^{br}$ with probability $1 - \epsilon_{i,t}$ and a random action from $\mathcal{A}_i(S_{i,t})$ with probability $\epsilon_{i,t}$ (uniformly distributed). In order to avoid learning an agent's neighbors' exploration strategy, only those experienced transitions were stored to the replay memory of the model-network, where the neighbor had exploited its knowledge. The probability of exploration $\epsilon_{i,t}$ as well as the learning rates α_t and β_t were all determined by the generic formula $s - ((1 - d^t) \cdot (s - e))$, where s is the start value, e is the end value and d^t is the decay factor d to the power of t. The selected parameters for these variables can be found in Table 4. Finally, as an implementation-specific sidenote, both the training of the Q-network and the model-network took place in a separate thread dissociated from the main thread.

6 RESULTS

The performance of deep-MARLIN was assessed by means of the average delay of all vehicles in the road network. To produce the results, 20 instances of the algorithm were simulated, each of them with a duration of three days (simulation time). Every second, the sum of the delay of all vehicles in the network was measured as the number of vehicles whose speed was below two meters per second and this value was then divided by the number of vehicles in the road network. This average vehicle delay was stored according to the first-in-first-out principle in a list of length 450. Every ten seconds, the average of this list was then stored in a second list. In addition, a moving average for the mean delay was formed based on the gathered data as described and over all 20 instances. This data was then plotted against the data from an implementation of MARLIN which was chosen to be the baseline algorithm. The implementation and benchmarking of MARLIN was similar to that of deep-MARLIN, i.e., only the state definition followed [6] because of the table-based representation of MARLIN. Since we did not have access to the exact signal timing sheet of Frankfurt am Main and since the authors of [6] already showed the superior performance of MARLIN when compared to a pre-timed algorithm, we decided to benchmark deep-MARLIN only against MARLIN.

Figure 2a and 2b show the average delay, where t indicates the time steps of training. The first graph in Figure 2a exemplarily shows the performance of one of the 20 MARLIN simulations. The average delay decreased to a stable low level and remained (nearly) unchanged beyond 100,000 training seconds. Hence, MARLIN was able to learn a policy that results in low and stable delay values.

The second graph in Figure 2a exemplarily shows the performance of one of the 20 deep-MARLIN simulations. The graph indicates more volatility in the average delay when compared to the previous result. This can be seen by the spikes in the graph for the average delay, which occurred at different time steps during training. These spikes can occur when ANNs are used for function approximation within a RL algorithm. Nevertheless, the algorithm also reached a constant level of average delay.

Figure 2b shows the smoothed average delay for both algorithms in one figure. Even if deep-MARLIN was more volatile, the algorithm converged to a policy with a lower average delay when compared to MARLIN. Furthermore, deep-MARLIN also converged faster towards this policy, which is assumed to be a (mixed) NE. This can be explained by the advantages ANNs can provide, i.e., by the ability to use larger and more informative state spaces and by the ability to generalize from experienced state-action pairs.

7 CONCLUSION

deep-MARLIN solves the deficiencies of the reviewed MARL frameworks without waiving their advantages. Due to the replacement of MARLIN's table-based representation of the joint state-action and model values by ANNs, deep-MARLIN enables coordinated decision making between multiple agents within large environments.

This was empirically shown by simulations where deep-MARLIN was applied to a large-scale ATLC problem with heterogeneous and decentralized agents. The results indicate that deep-MARLIN converged faster to a policy producing a lower average delay when compared to MARLIN. Furthermore, due to the use of ANNs as function approximators, there is no need for hand-crafted feature selection, i.e., raw sensory data or images of surveillance cameras can be used for the state representation which reduces the risk of selecting less informative features.

The proposed algorithm seem to be a promising solution to prevent traffic congestions in urban areas. It is also generic enough to be applied to similar problems, in which multiple co-existing agents may cause (further) non-stationarity in the environment which has to be captured for stabilizing the training process. The authors postpone this question to future research.

REFERENCES

[1] B. Abdulhai, R. Pringle, and G. J. Karakoulas. 2003. Reinforcement Learning for True Adaptive Traffic Signal Control. *Journal of Transportation Engineering* 129, 3 (2003), 278–285.

[2] I. Arel, C. Liu, T. Urbanik, and A. G. Kohls. 2010. Reinforcement Learning-Based Multi-Agent System for Network Traffic Signal Control. *IET Intelligent Transport Systems* 4, 2 (2010), 128–135.

[3] T. Başar and G. J. Olsder. 1982. *Dynamic Noncooperative Game Theory.* Academic Press Inc.

[4] L. Busconiu, R. Babuska, and B. De Schutter. 2008. A Comprehensive Survey of Multiagent Reinforcement Learning. *IEEE Transactions on Systems, Man, and Cybernetics, Part C (Applications and Reviews)* 38, 2 (2008), 156–172.

[5] T. Chu, J. Wang, L. Codecà, and Z. Li. 2019. Multi-Agent Deep Reinforcement Learning for Large-scale Traffic Signal Control. *IEEE Transactions on Intelligent Transportation Systems* (2019).

[6] S. El-Tantawy, B. Abdulhai, and H. Abdelgawad. 2013. Multiagent Reinforcement Learning for Integrated Network of Adaptive Traffic Signal Controllers (MARLIN-ATSC): Methodology and Large-Scale Application on Downtown Toronto. *IEEE Transactions on Intelligent Transportation Systems* 14, 3 (2013), 1140–1150.

[7] S. El-Tantawy, B. Abdulhai, and H. Abdelgawad. 2014. Design of Reinforcement Learning Parameters for Seamless Application of Adaptive Traffic Signal Control. *Journal of Intelligent Transportation Systems* 18, 3 (2014), 227–245.

[8] J. Gao, Y. Shen, J. Liu, M. Ito, and Shiratori N. 2017. Adaptive Traffic Signal Control: Deep Reinforcement Learning Algorithm with Experience Replay and Target Network. *arXiv:1705.02755* (2017).

[9] INRIX. 2019. INRIX 2018 Global Traffic Scorecard. http://inrix.com/scorecard/

[10] D. P. Kingma and J. L. Ba. 2014. Adam: A Method for Stochastic Optimization. *arXiv:1412.6980* (2014).

[11] L. Kuyer, S. Whiteson, B. Bakker, and N. Vlassis. 2008. Multiagent Reinforcement Learning for Urban Traffic Control Using Coordination Graphs. *Proceedings of the Joint European Conference on Machine Learning and Knowledge Discovery in Databases* (2008), 656–671.

[12] P. Lillicrap, T., J. Hunt, J., A. Pritzel, N. Heess, T. Erez, Y. Tassa, D. Silver, and D. Wierstra. 2015. Continuous Control with Deep Reinforcement Learning. *arXiv:1509.02971* (2015).

[13] L.-J. Lin. 1993. Reinforcement Learning for Robots Using Neural Networks. *Dissertation (Carnegie Mellon University Pittsburgh)* (1993).

[14] M. L. Littman. 1994. Markov Games as a Framework for Multi-Agent Reinforcement Learning. *Proceedings of the 11th International Conference on Machine Learning* (1994), 157–163.

[15] P. A. Lopez, M. Behrisch, L. Bieker-Walz, J. Erdmann, Y.-P. Flötteröd, R. Hilbrich, L. Lücken, J. Rummel, P. Wagner, and E. Wiener. 2018. Microscopic Traffic Simulation using SUMO. *Proceedings of the 21st IEEE International Conference on Intelligent Transportation Systems* (2018), 2575–2582.

[16] A. L. Maas, A. Y. Hannun, and A. Y. Ng. 2013. Rectifier Nonlinearities Improve Neural Network Acoustic Models. *Proceedings of the 30th International Conference on Machine Learning* (2013).

[17] V. Mnih, A. P. Badia, M. Mirza, A. Graves, T. Harley, T. P. Lillicrap, D. Silver, and K. Kavukcuoglu. 2016. Asynchronous Methods for Deep Reinforcement Learning. *Proceedings of the 33rd International Conference on Machine Learning* (2016).

[18] V. Mnih, K. Kavukcuoglu, D. Silver, A. Graves, I. Antonoglou, D. Wierstra, and M. Riedmiller. 2013. Playing Atari with Deep Reinforcement Learning. *arXiv:1312.5602* (2013).

[19] V. Mnih, K. Kavukcuoglu, D. Silver, A. A. Rusu, J. Veness, M. G. Bellemare, A. Graves, M. Riedmiller, A. K. Fidjeland, G. Ostrovski, S. Petersen, C. Beattie, A. Sadik, I. Antonoglou, H. King, D. Kumaran, D. Wierstra, S. Legg, and D. Hassabis. 2015. Human-level Control through Deep Reinforcement Learning. *Nature* 518, 7540 (2015), 529–533.

[20] R. Nair, P. Varakantham, M. Tambe, and M. Yokoo. 2005. Networked Distributed POMDPs: A Synthesis of Distributed Constraint Optimization and POMDPs. *Proceedings of the 20th National Conference on Artificial Intelligence* (2005), 133–139.

[21] N. Ono and K. Fukumoto. 1996. Multi-Agent Reinforcement Learning: A Modular Approach. *Proceedings of the 2nd International Conference on Multiagent Systems* (1996), 252–258.

[22] L. S. Shapley. 1953. Stochastic Games. *Proceedings of the National Academy of Sciences of the United States of America* 39, 10 (1953), 1095–1100.

[23] R. S. Sutton and A. G. Barto. 2018. *Reinforcement Learning: An Introduction.* The MIT Press.

[24] J. N. Tsitsiklis and B. Van Roy. 1997. An Analysis of Temporal-Difference Learning with Function Approximation. *IEEE Trans. Automat. Control* 42, 5 (1997), 674–690.

[25] C. J. C. H. Watkins. 1989. Learning from Delayed Rewards. *Dissertation (King's College London)* (1989).

[26] M. A. Wiering. 2000. Multi-Agent Reinforcement Learning for Traffic Light Control. *Proceedings of the 17th International Conference on Machine Learning* (2000), 1151–1158.

File name	randomTrips.py (SUMO package)
–seed, –end	timestamp (start time), 259200
-p,	259200/1500
–min-distance	450
-l, -L, –validate	/
–trip-attributes:	
departLane, arrivalLane	best, 0
departPos, arrivalPos	uniformly random, uniformly random
departSpeed, arrivalSpeed	0, 0

Table 1: Trip generation parameters

Size of the replay memories $\mathcal{M}^Q$	40960
Minimum size of $\mathcal{M}^Q$ to start the training	32
Size of the minibatches X^Q	32
Number of units in the input layer (IL)	74
Number of units in the first hidden layer (HL1)	400
Initial weights between IL and HL1 $\sim \mathcal{N}(\mu, \sigma)$	0, 0.1
Number of units in the second hidden layer (HL2)	200
Initial weights between HL1 and HL2 $\sim \mathcal{N}(\mu, \sigma)$	0, 0.1
Activation function for the units in IL, HL1, HL2	LeakyReLU [16]
Alpha value for the activation functions in IL, HL1, HL2	0.3
Number of units in the output layer (OL)	1
Activation function for the units in OL	Linear
Optimizer	Adam [10]
Loss function L^Q	Mean squared error

Table 2: deep-MARLIN Q-network architecture

Size of the replay memories $\mathcal{M}^\omega$	40960
Minimum size of $\mathcal{M}^\omega$ to start the training	32
Size of the minibatches X^ω	32
Number of units in the input layer (IL)	70
Number of units in the first hidden layer (HL1)	400
Initial weights between IL and HL1 $\sim \mathcal{N}(\mu, \sigma)$	0, 0.1
Number of units in the second hidden layer (HL2)	200
Initial weights between HL1 and HL2 $\sim \mathcal{N}(\mu, \sigma)$	0, 0.1
Activation function for the units in IL, HL1, HL2	LeakyReLU [16]
Alpha value for the activation functions in IL, HL1, HL2	0.3
Number of units in the output layer (OL)	3
Activation function for the units in OL	Soft-Max
Optimizer	Adam [10]
Loss function L^ω	Mean squared error

Table 3: deep-MARLIN model-network architecture

Discount factor γ	0.9
s_ϵ, s_α, s_β	1, 0.01, 0.01
d_ϵ, d_α, d_β	0.99995, 0.999975, 0.999975
e_ϵ, e_α, e_β	0.01, 0.0005, 0.0005

Table 4: deep-MARLIN parameters